

Analyzing the Second-Hand Car Market: Trends, Patterns, and Key Drivers

Dataset Source: Original

Smarandi Anastasia, 411
Gherghisan Sebastian Mihai, 411
Orghidan Radu, 411
Isfan Mihai George, 411

Final Project - Big Data
Faculty of Mathematics and Computer Science - University of Bucharest
June 2025

Dataset Description: Used Vehicle Listings in Romania

Our dataset comprises structured information on used vehicle listings collected from three major online automotive marketplaces in Romania: **OLX**, **Publi24**, and **Autovit**. The data was **ethically crawled**, adhering to platform guidelines and legal standards.

The dataset captures a comprehensive snapshot of the Romanian second-hand vehicle market and includes detailed attributes for each listing. These attributes span a wide range of categories:

- **Vehicle Specifications:** brand, model, version, year of manufacture, engine power and displacement, fuel type, transmission, drivetrain, body type, and number of doors and seats.
- **Market Information:** price, currency, negotiability, seller type (individual or dealer), and warranty details.
- **Legal and Environmental Data:** first registration date, number of previous owners, pollution standard (e.g., Euro 6), CO₂ emissions, particulate filter status, and origin country.
- **Technical and Visual Attributes:** mileage, service history, color, metallic paint, tuning, and image availability.
- **Geographical Indicators:** location, county, postal code, and source site ID.

- **Descriptive Fields:** textual descriptions, equipment lists, and links to the original advertisement.
- **Metadata:** unique listing IDs, dates of posting and ingestion, and auxiliary fields for data management and provenance.

This rich, multi-source dataset enables robust analysis using big data techniques such as dimensional reduction methods (PCA, UMAP and t-SNE).

Dataset Preparation and Exploratory Data Analysis

To better understand the characteristics of the second-hand car market, we performed an in-depth exploratory data analysis (EDA) on the dataset. The aim was to prepare the data for downstream tasks (regression and clustering) and uncover initial insights into the structure and distribution of the data.

Upon loading the dataset, we examined its basic structure and identified potential data quality issues:

- Missing values in both numerical and categorical fields.
- Inconsistent data formatting (e.g., string representations of missing values like "\N").
- Heterogeneous currencies (RON and EUR), requiring conversion for meaningful price comparisons.

To prepare the dataset for analysis and modeling, we implemented a custom preprocessing pipeline:

- **Feature Selection:** We retained the most relevant numerical and categorical features based on domain knowledge:
 - *Numerical:* manufacturing_year, engine_hp, engine_cc, km_no, price, doors_no
 - *Categorical:* make, fuel_type, transmission_type, body
- **Handling Missing Values:**
 - For numerical columns, we used median imputation.
 - For categorical columns, the most frequent value was imputed.
- **Currency Normalization:** Prices in RON were converted to EUR by dividing by an approximate exchange rate of 5. Entries in other currencies were dropped or marked as invalid.
- **Outlier Removal:** We applied IQR filtering to detect and exclude extreme outliers in key numerical columns (engine_hp, engine_cc, km_no, price), which would otherwise distort the analysis.
- **Encoding:** Categorical variables were one-hot encoded.
- **Scaling:** Numerical features were standardized using StandardScaler.

After preprocessing, we obtained a clean dataset with both raw and transformed versions available for EDA and further modeling.

Numerical Feature Analysis

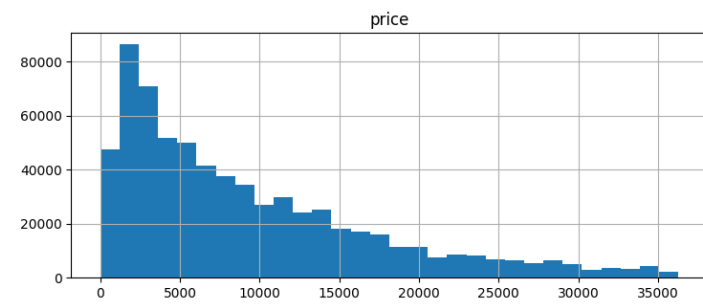
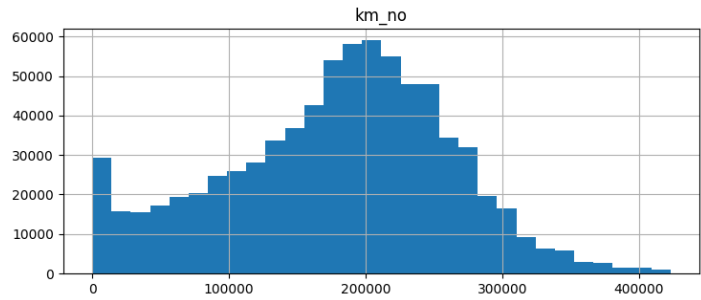
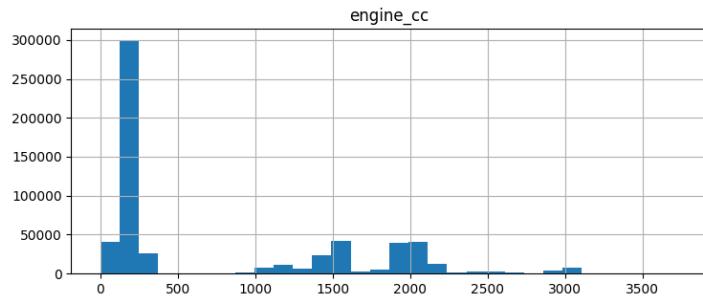
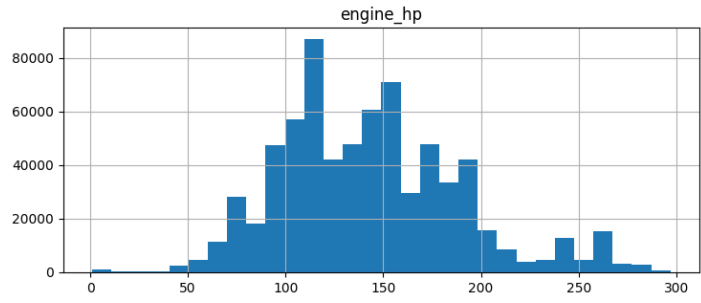
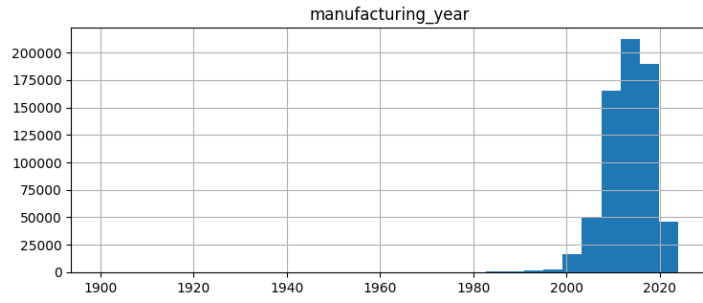
We conducted descriptive and visual analyses on the numerical variables to understand their distribution, spread, and relationships:

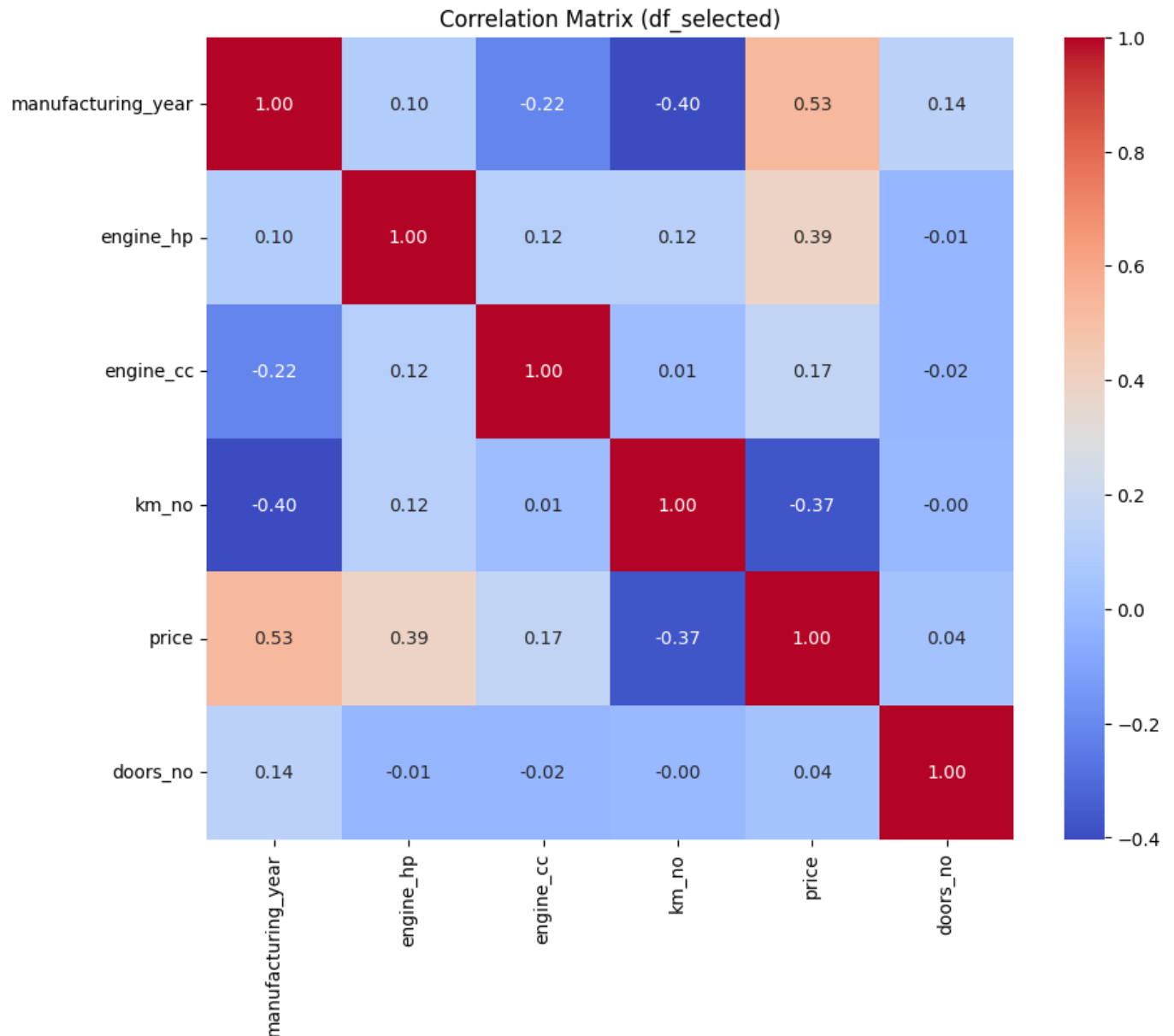
- **Descriptive Statistics:** We examined measures such as mean, median, and standard deviation. As expected, km_no and price displayed right-skewed distributions, with a long tail of high-mileage, high-priced vehicles.
- **Histograms:** Histograms were plotted to visualize the spread of values across numerical features. This revealed clusters of cars around common engine sizes (e.g., ~1.6L and ~2.0L), and notable peaks in manufacturing_year corresponding to newer models.
- **Correlation Matrix:** A heatmap of Pearson correlations between numerical variables highlighted meaningful relationships:
 - Strong negative correlation between km_no and price, reflecting that higher mileage typically lowers resale value.
 - Moderate positive correlation between engine_hp/engine_cc and price, suggesting performance characteristics influence pricing.

Summary statistics for numerical features:

Feature	Count	Mean	Std Dev	Min	25%	50%	75%	Max
Manufacturing Year	685,136	2013.14	4.95	900	2010	2014	2017	2024
Engine HP	704,063	141.81	46.79		110	140	170	297
Engine CC	579,102	774.79	830.40		160	200	1595	3733
Kilometers (km_no)	765,252	178,645	83,147		124,015	188,500	236,000	423,000
Price (€)	671,830	9,294.26	7,977.48		2,950	6,950	13,490	36,271
Doors Number	532,987	4.68	0.63		4	5	5	6

Histograms of Numerical Features (df_selected)





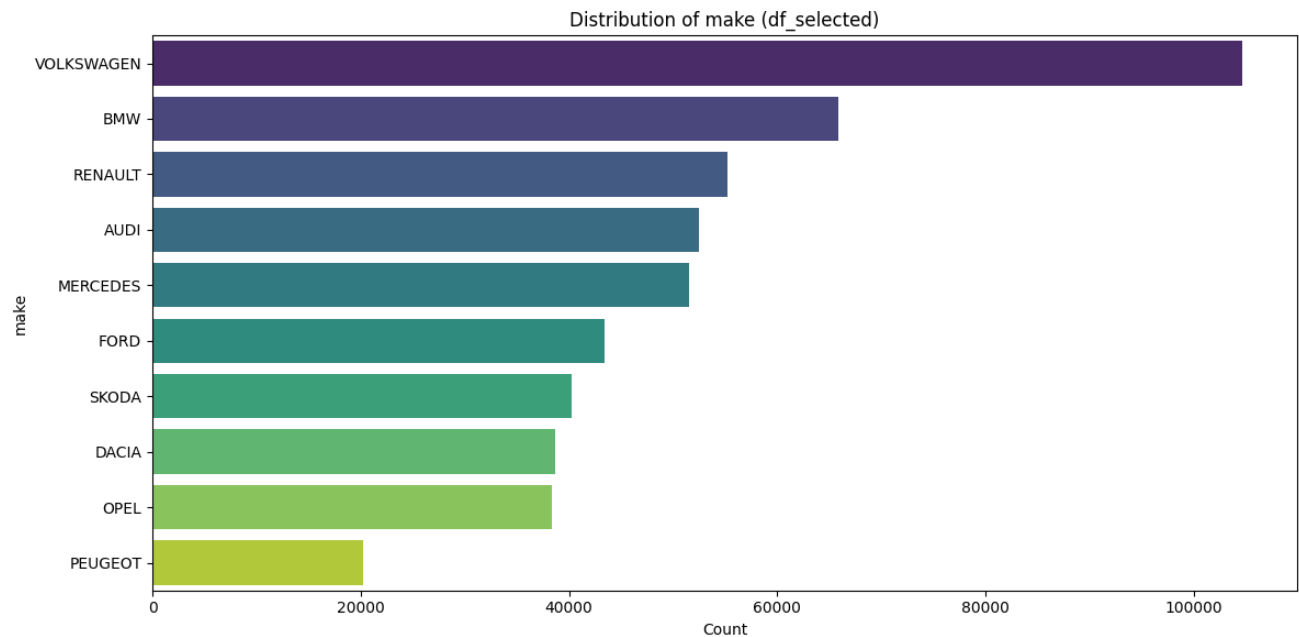
Categorical Feature Analysis

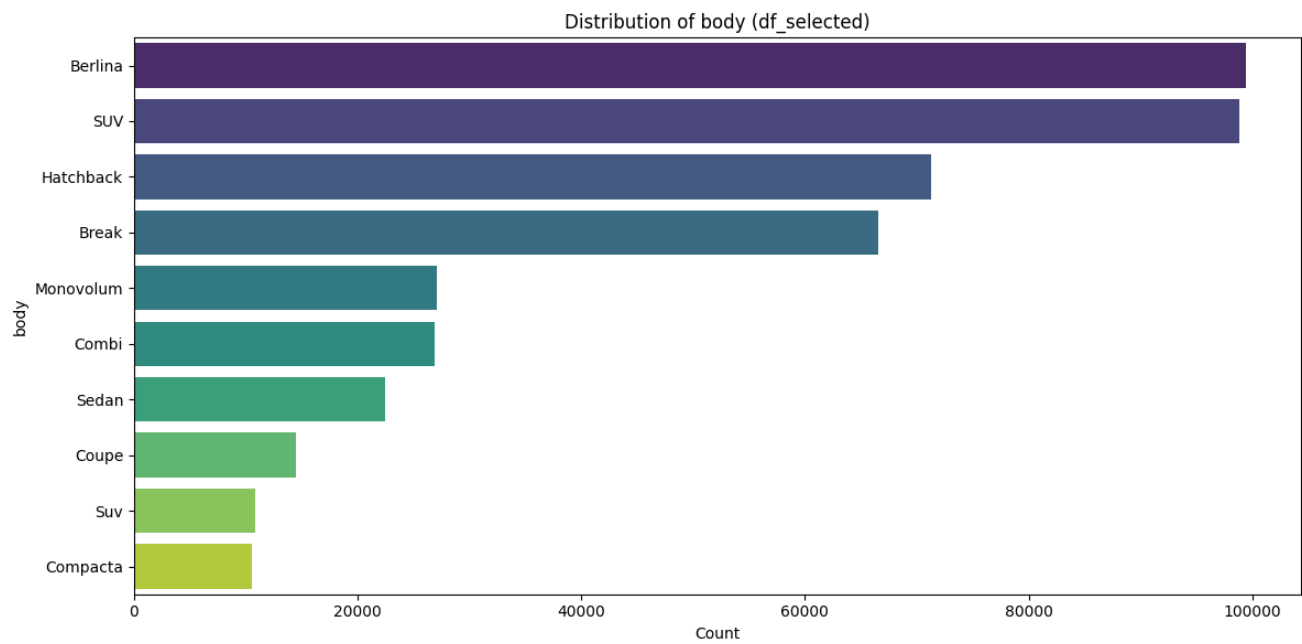
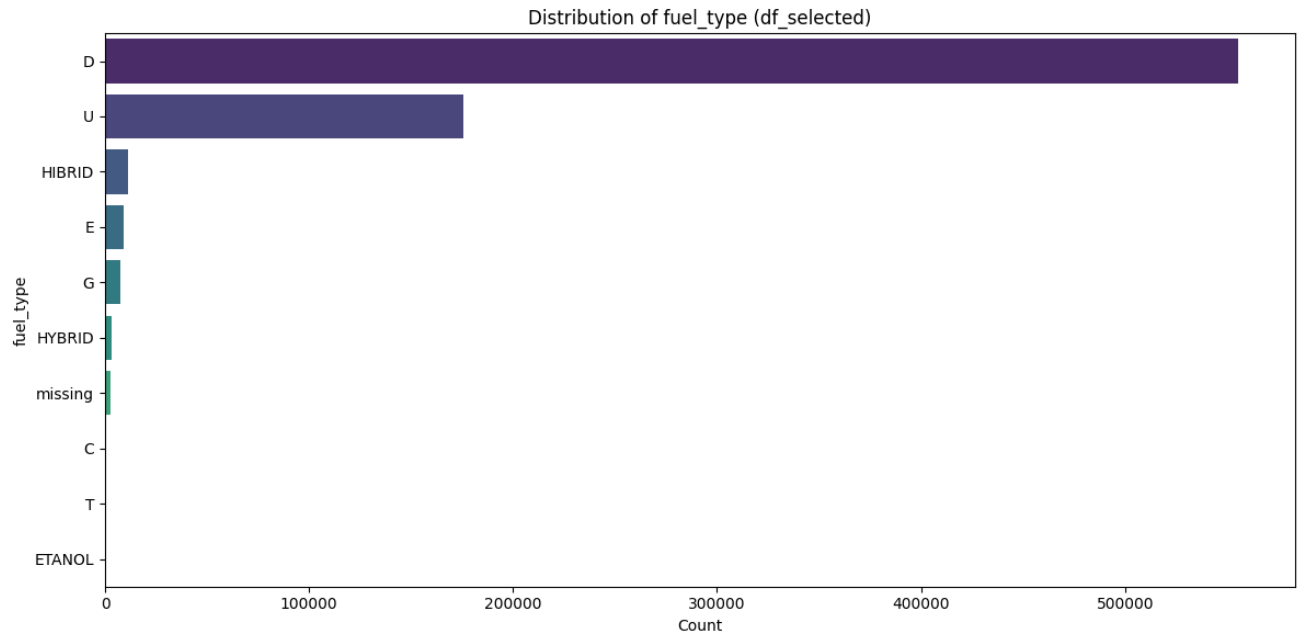
Categorical variables were analyzed through value counts and count plots to identify popular segments:

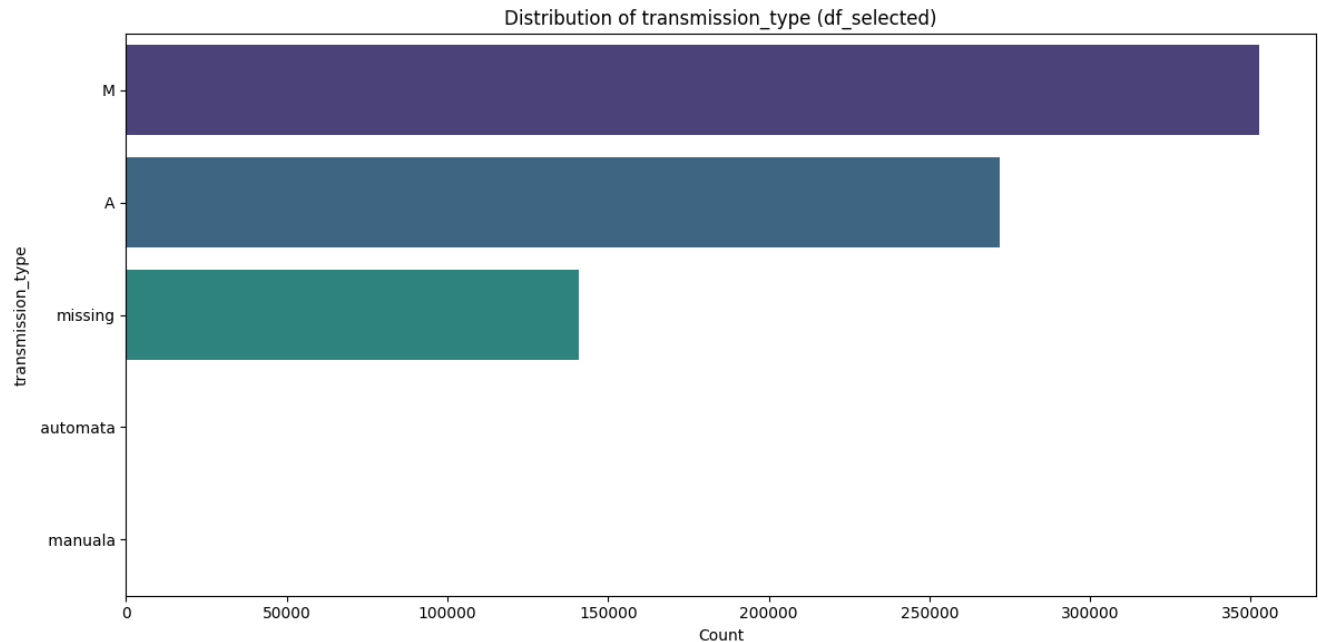
- **Top Car Makes:** Brands like BMW, Audi, Volkswagen, and Mercedes dominated the dataset, aligning with expectations for the European second-hand market.
- **Fuel Type & Transmission:** Petrol remained the most common fuel type, with automatic transmissions gaining popularity but still less prevalent than manual ones.

- **Body Types:** Sedans and hatchbacks were the most frequent, while SUVs appeared in significant numbers, reflecting current market trends.

Visualizations of category frequencies (limited to the top 10 values per feature) provided a quick overview of prevalent models and configurations in the dataset.







Dimensional reduction

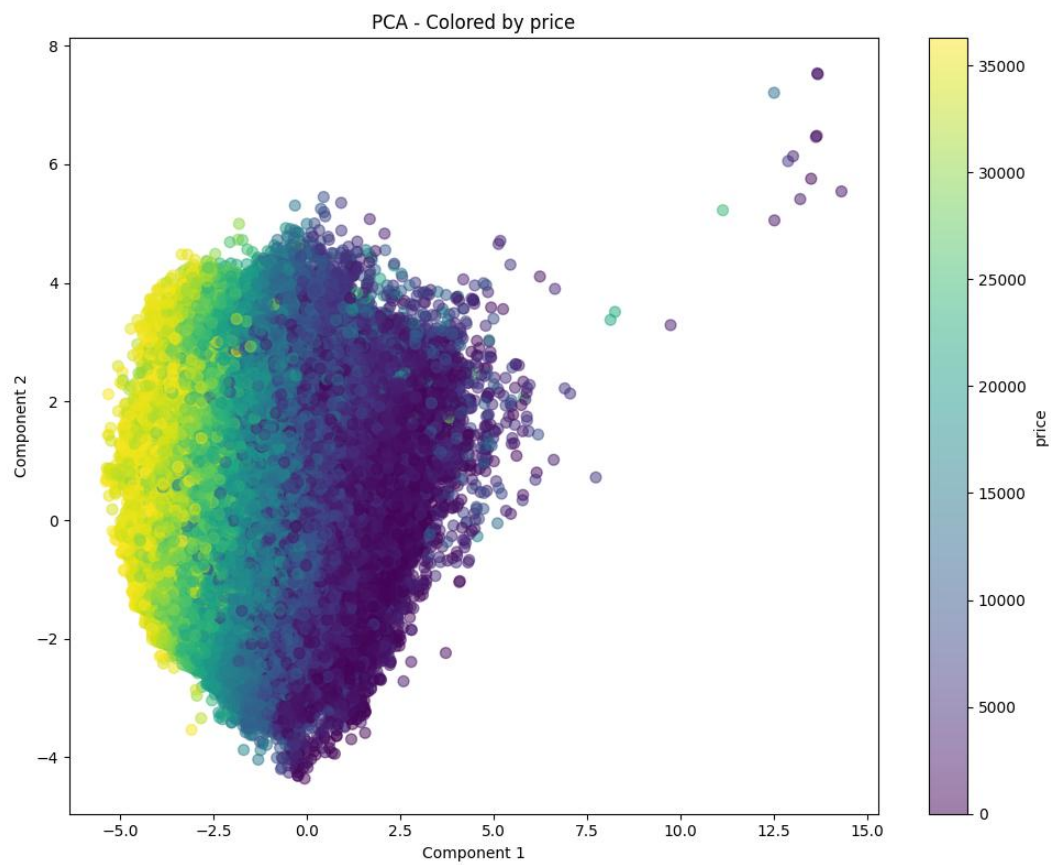
Three distinct mathematical approaches drive the analysis. **Principal Component Analysis** identifies linear combinations of features that capture maximum variance, providing interpretable components that reveal primary data trends. **t-Distributed Stochastic Neighbor Embedding** preserves local neighborhood relationships through probability-based optimization, effectively revealing cluster structures and groupings within the dataset. **Uniform Manifold Approximation and Projection** balances both local and global structural preservation using manifold learning theory, offering comprehensive relationship visualization.

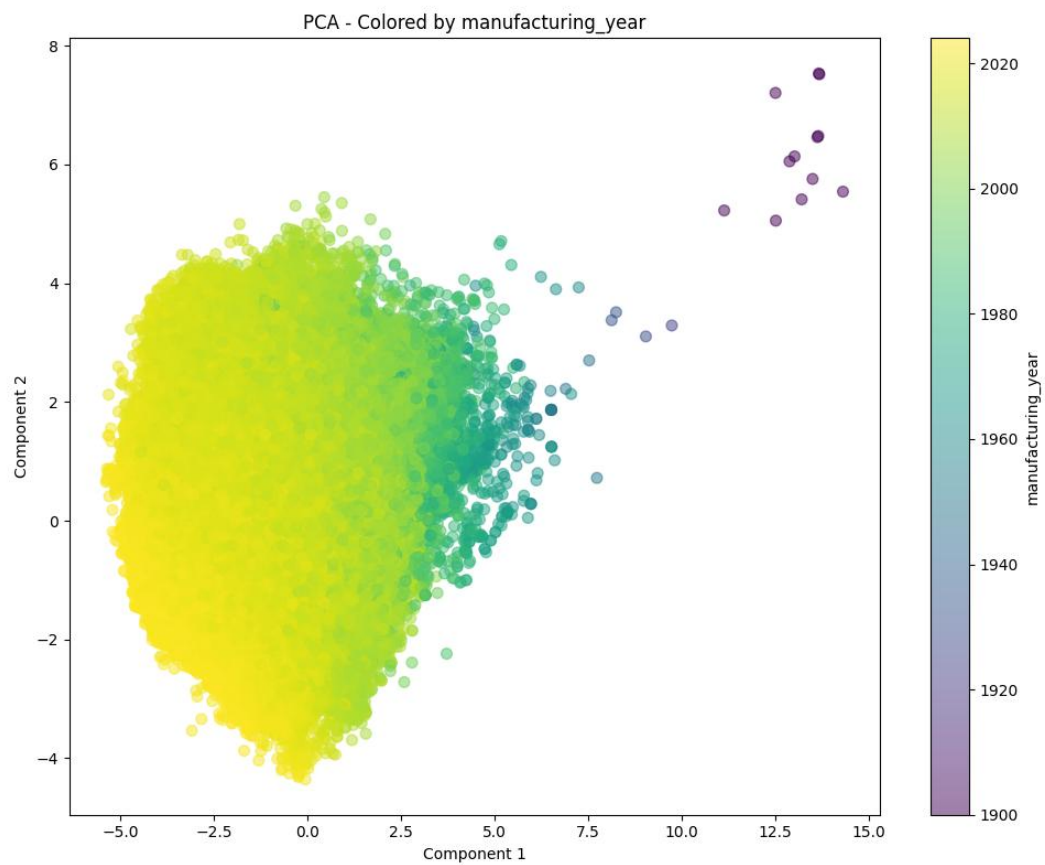
Preprocessing

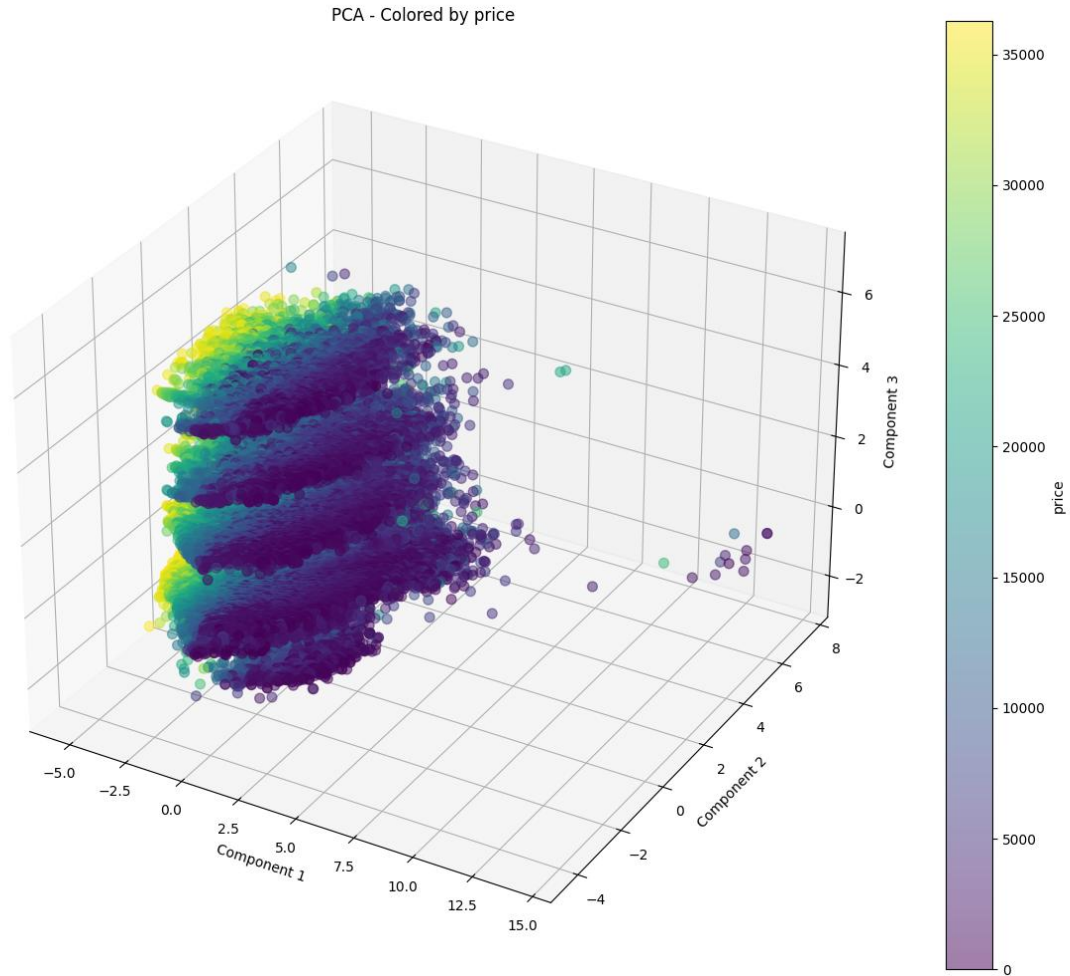
Categorical features require transformation into numerical format since machine learning algorithms operate exclusively on numerical data. Our implementation employs one-hot encoding, a technique that converts each categorical variable into multiple binary columns. For example, a "make" column containing values like "Toyota," "Honda," and "Ford" becomes three separate binary columns: "make_Toyota," "make_Honda," and "make_Ford," where each row contains 1 for the applicable manufacturer and 0 for all others.

PCA

Principal Component Analysis represents the foundational approach to dimensionality reduction through linear transformation techniques. This method identifies the primary directions of variance within high-dimensional datasets, creating new variables called principal components that capture the most significant patterns in the data. PCA excels at revealing underlying trends and relationships by projecting complex data onto lower-dimensional spaces while preserving maximum information content.







Component 1 captures ~61% of the variance and represents the primary way cars differ from each other. Moving on the x axis, high C1 values correspond to entries of high mileage car, manual transmission, older cars while the low C1 values represent newer manufacturing years, higher horsepower and automatic transmissions

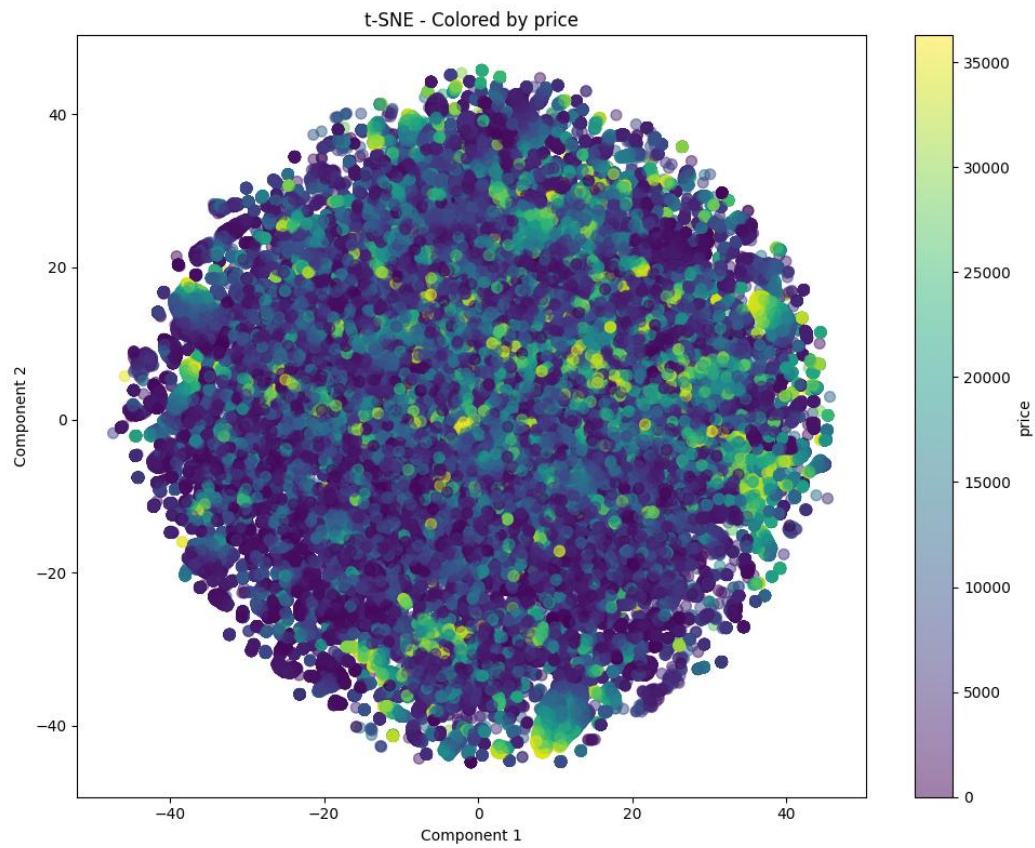
Component 2 is moving on the y axis and high C2 values correspond to high-performance vehicles with more horsepower, high mileage cars, larger engines, diesel and automatic transmission. Low C2 values represent newer cars, manual transmission.

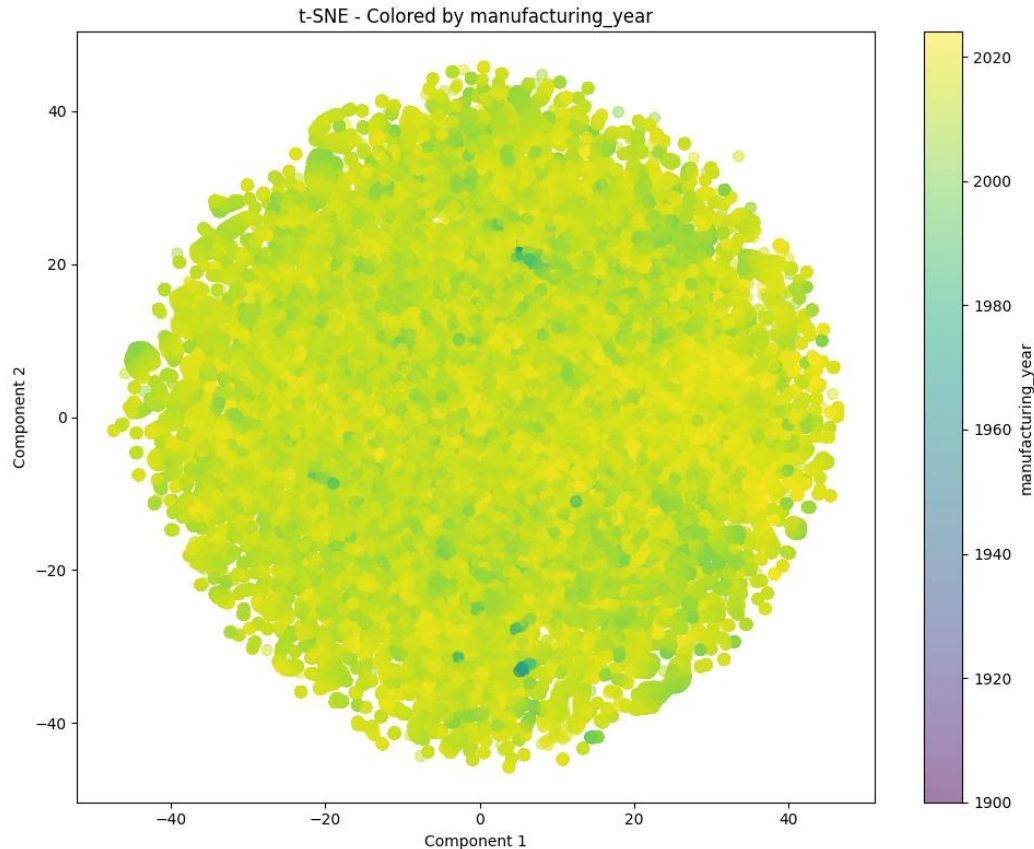
Component 3 is dominated by the number of doors a car has. So, C1 essentially captures the "premium new car vs. budget used car" spectrum while C2 captures "heavy-duty/high-performance vs. efficient/modern" characteristics. High PC2 suggests work vehicles, performance cars, or well-used vehicles with powerful engines.

t-SNE

t-Distributed Stochastic Neighbor Embedding specializes in preserving local neighborhood relationships within high-dimensional data through sophisticated probability-based modeling. This non-linear technique focuses on maintaining the relative similarities between

closely related data points, resulting in visualizations that effectively reveal cluster structures and groupings that may remain hidden in linear approaches.

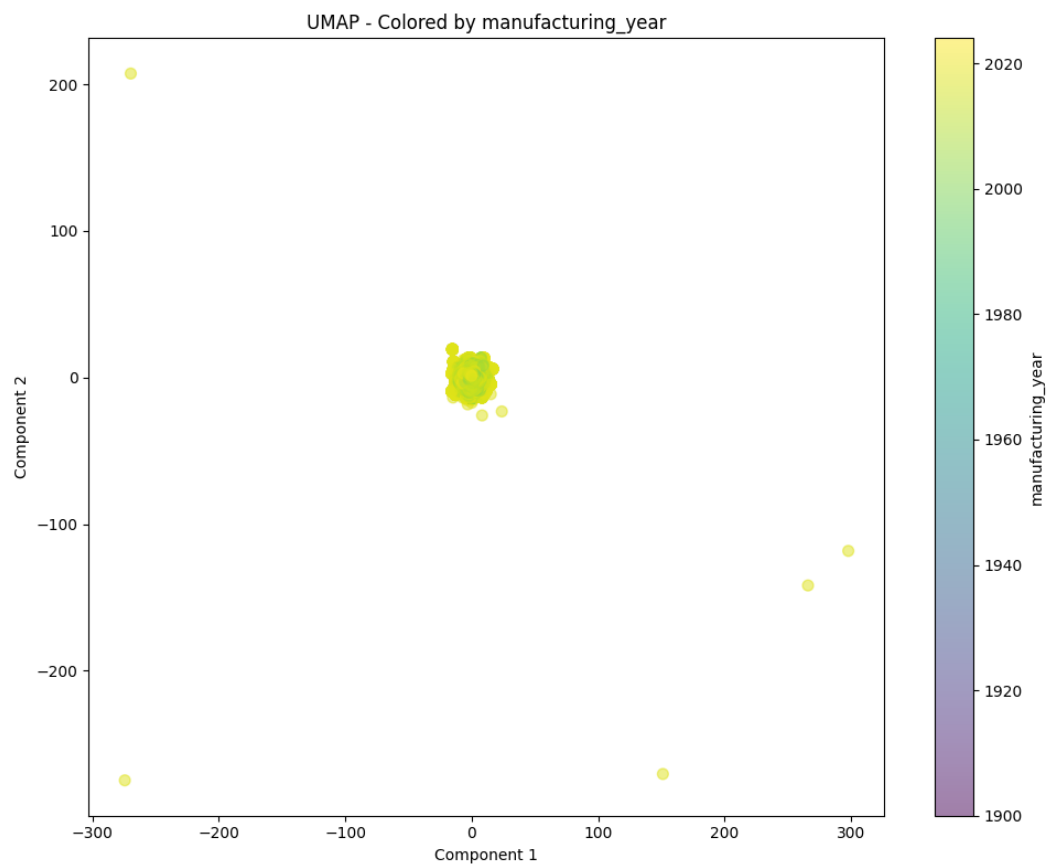


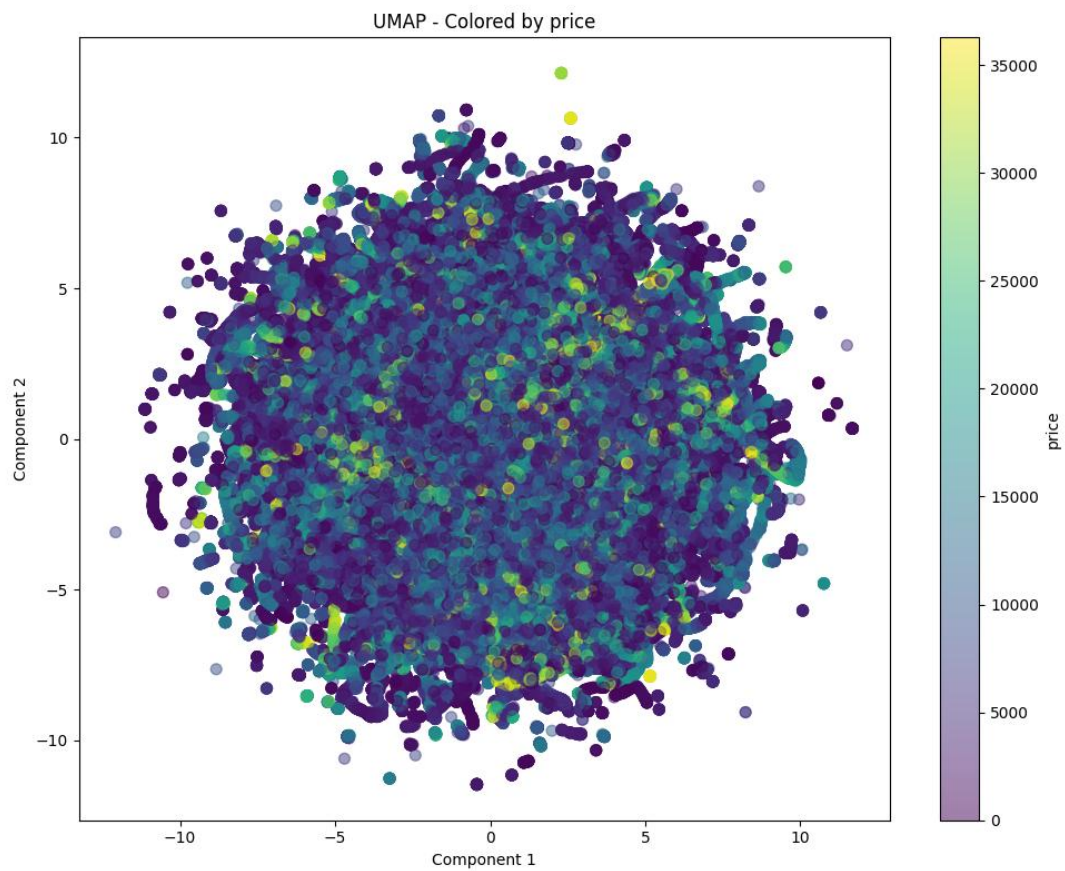


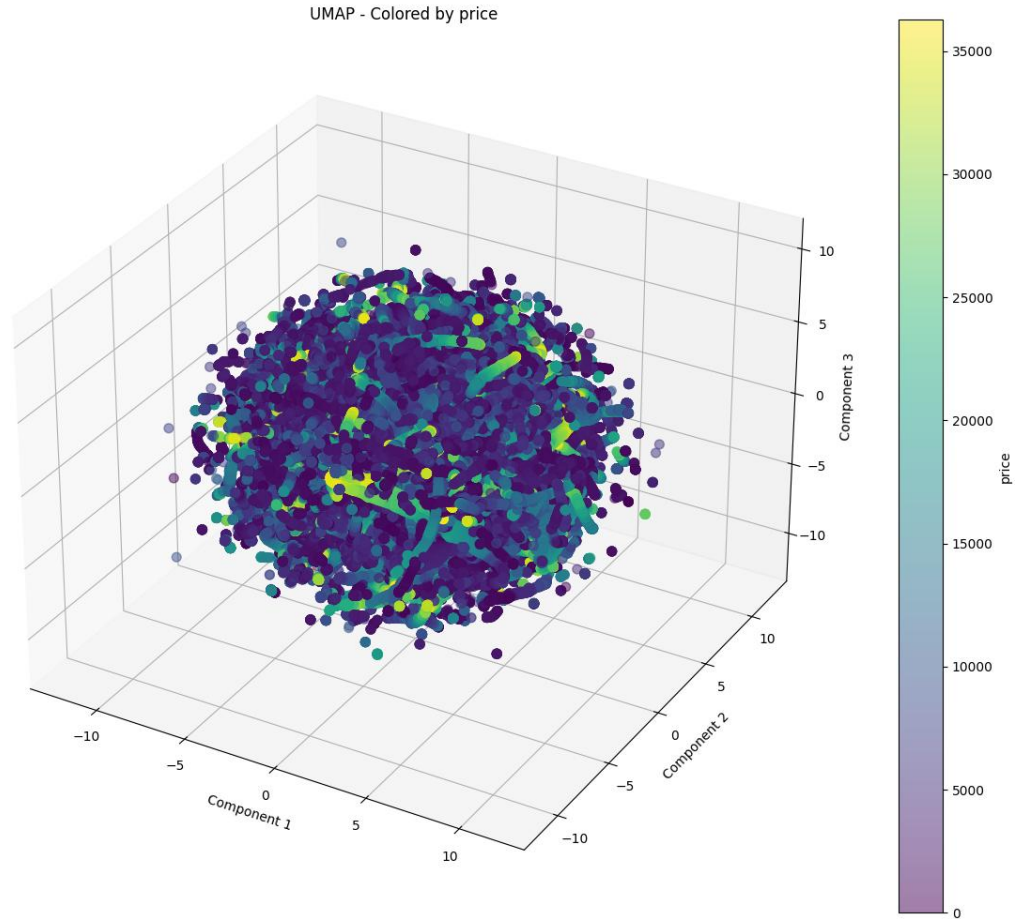
Dimension 1 captures a separation between diesel and petrol cars and manual transmissions from automatics. Dimension 2 represents engine displacement and door count. t-SNE reveals distinct vehicle categories based on powertrain and configuration choices, creating tighter clusters than PCA's linear projections.

Uniform Manifold Approximation and Projection

UMAP represents the most advanced dimensionality reduction technique, combining the benefits of both local structure preservation and global relationship maintenance through manifold learning theory. This method creates balanced visualizations that retain both fine-grained clustering information and broader structural patterns within the dataset. UMAP demonstrates superior computational efficiency compared to t-SNE while providing more nuanced representations of data relationships. The technique excels in revealing hierarchical structures and maintaining proportional distances between different data regions.







Unlike PCA's linear projections or t-SNE's local clusters, UMAP reveals how different car characteristics form a type of interconnected market system.

Component 1 represents the fuel type and regional market positioning. It shows that certain car brands and body types are preferred when choosing a diesel car for example. Judging from the date, Volkswagen shows to be the predominant diesel manufacturer that people choose.

Component 2 represents vehicle type and customer segmentation. Features like door configuration, transmission types, performance indicators and body style are correlated when buying a car. For example, on one end of the dimension we have utilitarian multi door vehicles with manual transmissions and on the other end we have 2 door high engine displacement, high horsepower cars.

Component 3 represents engine performance and market positioning.

To monitor the performance of applying those dimensionality reduction algorithms, we measured the computation times for each method using GPU acceleration. All experiments were conducted on the GPU offered by Google Colab (T4 GPU), leveraging RAPIDS cuML implementations for PCA, t-SNE, and UMAP. The results were the following:

Algorithm	Computation Time (seconds)
PCE	0.5

t-SNE	29.6
UMAP	53.8

The PCA method had a near-instant execution (0.5 seconds), making it highly practical even for massive datasets. Both t-SNE and UMAP, which are typically computationally expensive on CPU (took way too long to complete the algorithms), but had benefited significantly from GPU acceleration, with t-SNE reducing to 29.6 seconds and UMAP to 53.8 seconds.

Models

Regression Models

Regression models are fundamental tools in machine learning and statistics, allowing us to establish relationships between dependent variables (target) and independent variables (features). Below, we outline the models used, explaining their core characteristics and why they were selected for our study.

Linear Regression is a simple yet powerful baseline model that assumes a linear relationship between the independent variable(s) (X) and the dependent variable (y). It is easy to implement and interpret. Despite its simplicity, it serves as an excellent starting point for evaluating relationships in datasets.

XGBoost (Extreme Gradient Boosting) is a highly efficient and powerful gradient boosting algorithm designed for performance and scalability. It effectively handles non-linear relationships and includes advanced features to mitigate overfitting. XGBoost's ability to optimize computational resources while delivering high accuracy makes it a top choice for large-scale and complex datasets.

Gradient Boosting is a machine learning technique that combines multiple weak learners, typically decision trees, to build a strong predictive model. By sequentially adding trees that correct the errors of the previous models, it minimizes the loss function over iterations. However, it can be slower on large datasets compared to modern variants like XGBoost or LightGBM (a grid search on LightGBM took more than an hour and it didn't even finished the search, when the grid search for XGB and GB took just 10-15 minutes), making it more suited to smaller-scale problems or where computational efficiency is less critical.

The main regression task that has been focused on this project, was predicting the **Price** for used cars, based on a variety of technical and categorical features extracted from car advertisements. To prepare the data for modeling, a preprocessing and dimensionality reduction pipeline was implemented, which included feature selection, handling of missing values, encoding of categorical variables, scaling, and finally, dimensionality reduction using both Principal Component Analysis (PCA) and UMAP.

The main features used for prediction were: manufacturing_year, engine_hp, engine_cc, km_no, doors_no, make, fuel_type, transmission_type and body. After preprocessing, the resulting feature matrix was reduced to a lower-dimensional representation using PCA and UMAP, in order to mitigate the curse of dimensionality and to allow for more efficient model training. Each of the used models (Linear Regression, GB and XGB) was trained and tested both on the PCA-reduced data and on the UMAP-reduced data. The performance was measured using

several standard regression metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), R² Score, and Adjusted R². Also, for all of the models, the data has been splitted into 80% training and 20% testing, in order to evaluate the model.

For the tree-based models (GBR and XGBR), we performed hyperparameter optimization using RandomizedSearchCV, exploring a wide range of parameter values to find the optimal configuration.

The hyperparameters tuned for XGBoost were the following:

n_estimators	100, 150, <u>200</u>
max_depth	3, 5, <u>7</u>
learning_rate	0.03, 0.05, <u>0.1</u>
subsample	<u>0.8</u> , 1
colsample_bytree	0.8, <u>1</u>
gamma	0, 0.1, <u>0.3</u> , 1
reg_alpha	0, 0.1, <u>1</u> , 10
reg_lambda	1, <u>3</u> , 10
min_child_weight	1, <u>3</u> , 5

The difference in time between the training with the raw data, vs PCA and UMP is pretty significant. More exactly, Using the raw dataset, training required about **11.1 seconds**, but with PCA this dropped to just **2.0 seconds**, and with UMAP to about **2.4 seconds** which points out that reducing the data set with those techniques, had a positive impact over the training time.

The highlighted parameters have been selected for both PCA and UMAP dimension reduction. Although the hyperparameters were the same, the results were a bit different

Test	PCA	UMAP
MSE	1721931894.15	689439332.01
RMSE	41496.17	26257.18
MAE	4758.09	10227.52
R^2	0.6655	0.8661
Adjusted R^2	0.6655	0.8661

Although the same hyperparameter configuration was selected for both PCA and UMAP-reduced datasets, the predictive results differed considerably between the two approaches. XGBoost performed noticeably better on the UMAP-reduced data, achieving a higher R^2 score (0.8661 vs. 0.6655) and substantially lower error metrics (MSE and RMSE) compared to PCA. This suggests that, in this particular case, UMAP was able to generate a data representation that preserved more information relevant for the price prediction task, allowing the XGBoost model to better capture the underlying patterns and relationships in the data.

The hyperparameters tuned for Gradient Boosting were:

n_estimators	100, <u>150</u> , 200
max_depth	3, 5, <u>7</u>
learning_rate	0.03, 0.05, <u>0.1</u>
subsample	0.8, <u>1</u>
min_samples_split	<u>2</u> , 5, 10
min_samples_leaf	<u>1</u> , 3, 5
max_features	<u>'sqrt'</u> , 'log2

The highlighted parameters have been selected for PCA dimensionality reduction and gave the following results:

Test	PCA
MSE	88250926.62
RMSE	9394.20
MAE	4505.84
R^2	0.9829
Adjusted R^2	0.9829

n_estimators	<u>100</u> , 150, 200
--------------	-----------------------

max_depth	3, 5, <u>7</u>
learning_rate	0.03, 0.05, <u>0.1</u>
subsample	0.8, <u>1</u>
min_samples_split	2, 5, <u>10</u>
min_samples_leaf	1, 3, <u>5</u>
max_features	'sqrt', ' <u>log2</u>

The hyper parameters chosen for UMAP dimensionality reduction were the ones above, and the following results:

Test	UMAP
MSE	2402013832.67
RMSE	49010.34
MAE	12251.66
R^2	0.5334
Adjusted R^2	0.5334

As for the time obtained while using the raw, PCA and UMAP data, there isn't such a big difference for the Gradient Boosting model. Training on raw features took about **169.7 seconds**, while PCA and UMAP both required slightly more time, approximately **179.3** and **181.5 seconds**, respectively. Overall, this suggests that for Gradient Boosting, the computational cost is dominated by the algorithm's complexity and less by the dimensionality of the feature space.

Although the grid search explored the same range of hyperparameters for both PCA and UMAP-reduced datasets, the best performing parameter combination and model performance differed significantly. For the PCA-reduced data, Gradient Boosting achieved an R^2 of 0.9829 and very low RMSE, indicating that this feature representation was highly suitable for the model. In contrast, the best model on UMAP-reduced data (with different optimal hyperparameters) achieved a much lower R^2 of 0.5334 and much higher errors. This suggests that PCA preserved more of the information relevant to Gradient Boosting for this regression task, whereas UMAP's non-linear reduction, although powerful for other models, did not produce a feature space as well suited for this particular regressor.

The last model used for this regression task, was the Linear Regression, used as a baseline to provide a reference point for evaluating the performance for the other 2 models. This model had the worst results:

Test	PCA	UMAP
MSE	3780659279.78	5146903146.41
RMSE	61487.07	71741.92
MAE	22449.14	13499.69
R ²	0.2657	0.0003
Adjusted R ²	0.2656	0.0003

Lastly, the training times obtained for Linear Regression model while using the 3 versions of data, points out that this is the only model that had a real visible time reduction while using the dimensionality reduced data. Using the raw data, training completed in about **8.57 seconds**, while with PCA and UMAP-reduced data, the times dropped further, to less than **0.11 seconds**.

All in all, for this regression task, the more complex models achieved significantly better performance for this regression task. An interesting aspect for this task was the effectiveness of the dimensionality reduction technique proved to be highly model-dependent: UMAP yielded the best results when used with XGBoost, while PCA was much better suited for Gradient Boosting.

Regarding the training time, dimensionality reduction using PCA or UMAP can result in considerable efficiency increases in training time for complicated models such as XGBoost, decreasing computation from over 11 seconds to as little as 2 seconds. Gradient Boosting's feature reduction had less of an influence on training speed, probably due to the algorithm's construction and evaluation of trees. Dimensionality decrease makes Linear Regression training virtually quick, despite the fact that it is already fast. In conclusion, PCA and UMAP can significantly cut training time for some models, particularly boosting approaches, although the effect varies depending on the methodology employed.

Clustering Models

To explore the underlying structure of the dataset and identify groups of similar vehicle listings, unsupervised clustering techniques were applied. Specifically, **K-Means** and **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) were selected due to their complementary strengths in partitioning data.

K-Means is a centroid-based algorithm that partitions the data into k clusters by minimizing the variance within each cluster. It assumes spherical cluster shapes and requires the number of clusters to be specified in advance. While efficient and widely used, it is sensitive to the initial choice of centroids and does not perform well with clusters of varying densities or non-globular shapes.

In contrast, **DBSCAN** is a density-based algorithm that groups together data points that are closely packed, while marking points in low-density regions as outliers. It does not require the number of clusters to be predefined and is particularly effective at discovering clusters of arbitrary shape and handling noise in the dataset.

To evaluate and compare the clustering performance of both algorithms, the Silhouette Score was used. This metric assesses how similar an object is to its own cluster compared to other clusters, with higher scores indicating better-defined and more cohesive clusters. Data was sampled from the original dataset in order to navigate the limited computational resources we had access to.

Hyperparameters tuned for K-Means:

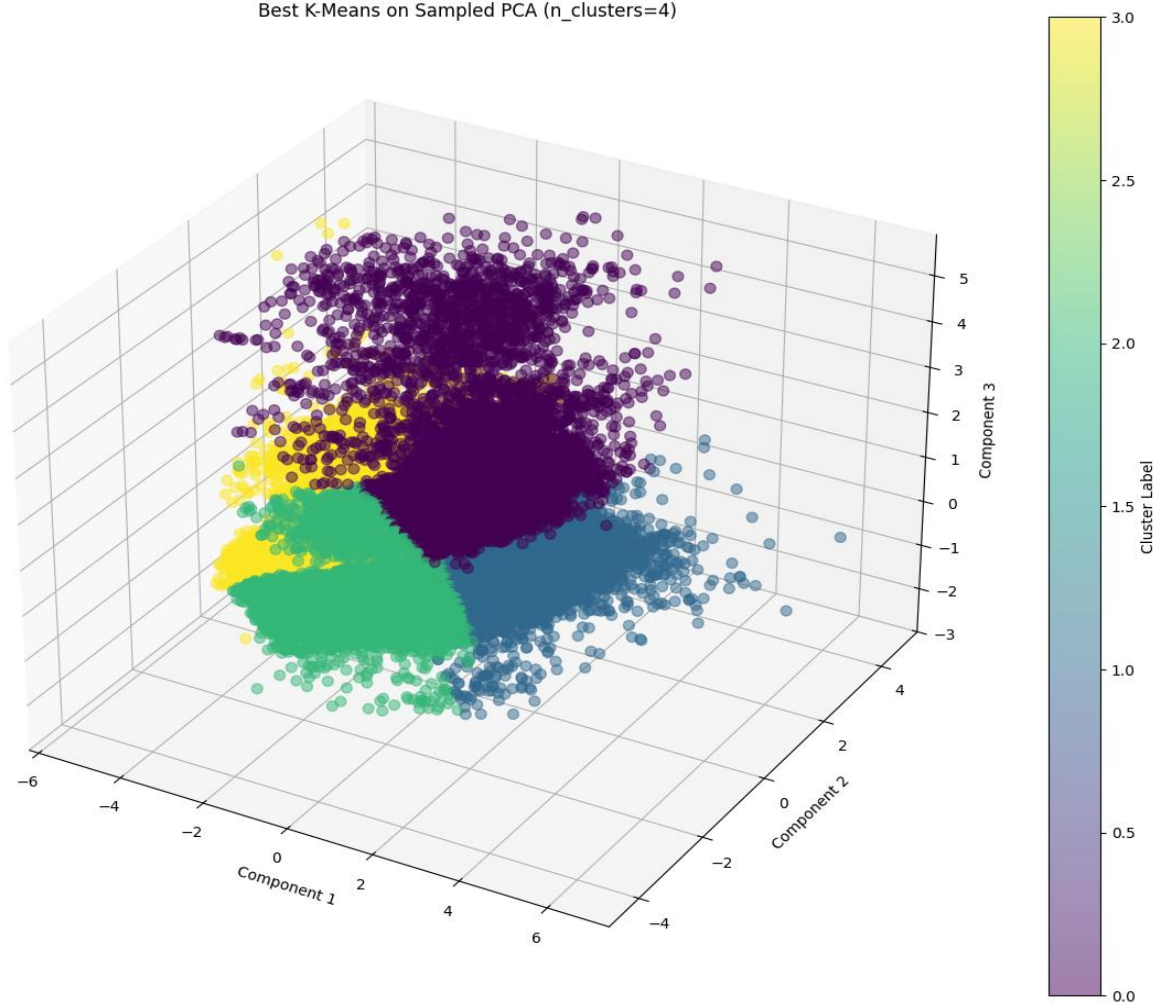
n_clusters	1, 2, 3, 4, <u>5</u> , 6, 7
------------	-----------------------------

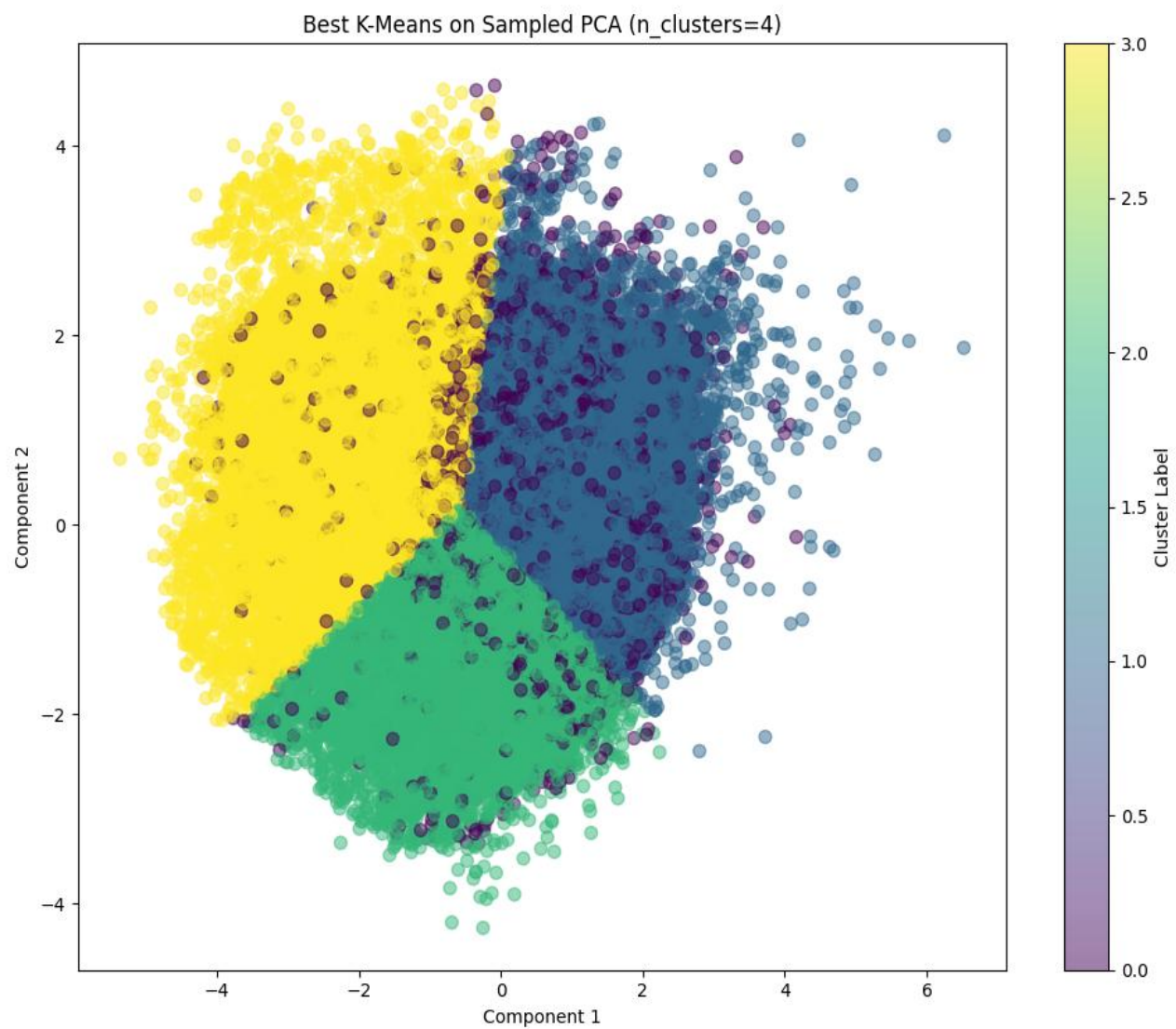
Hyperparameters tuned for DBSCAN:

eps	0.5, 1.0, <u>1.5</u>
min_samples	5, <u>10</u>

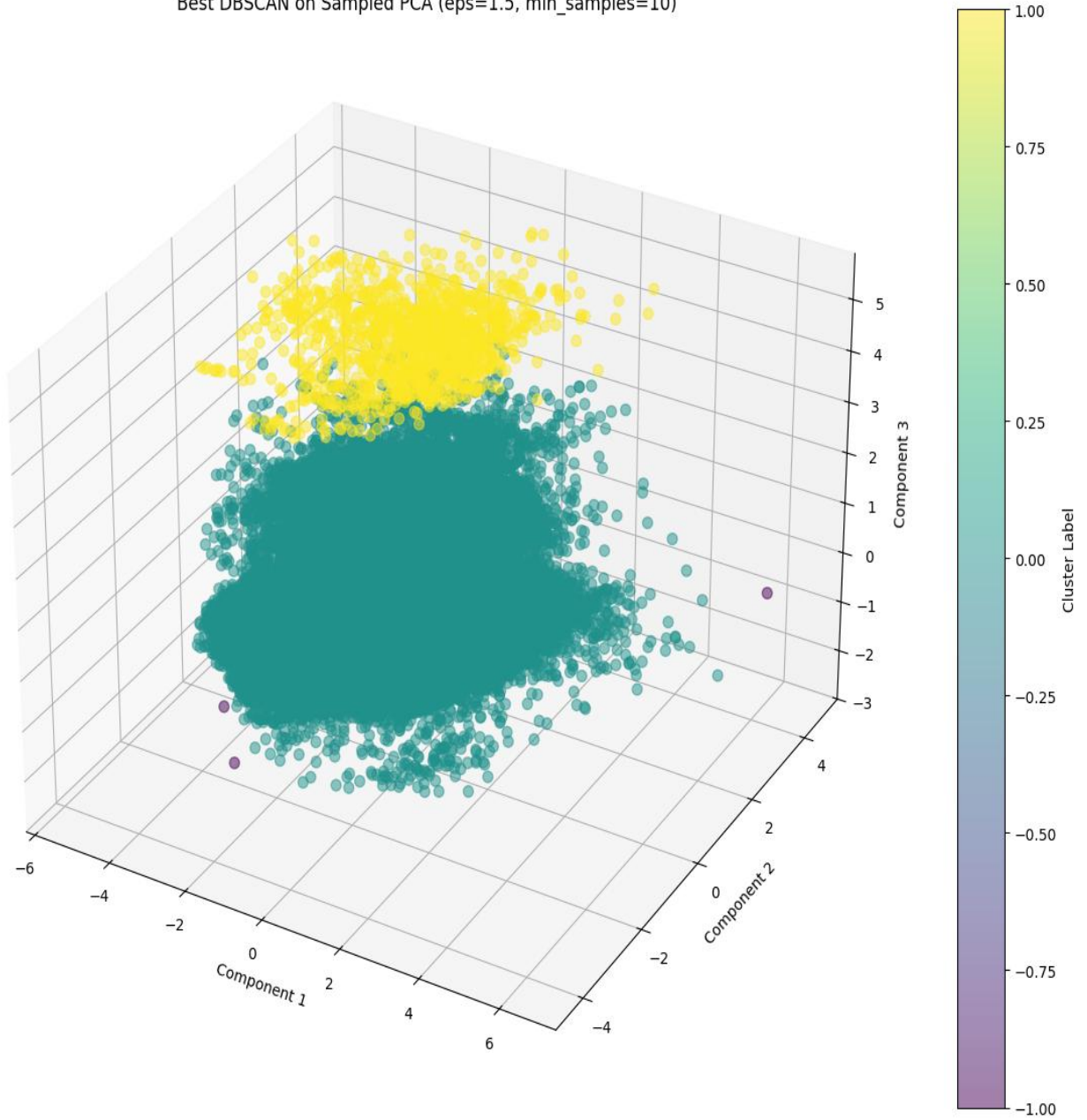
For the best DBSCAN result, we obtained **2 clusters** and **3 noise points**.

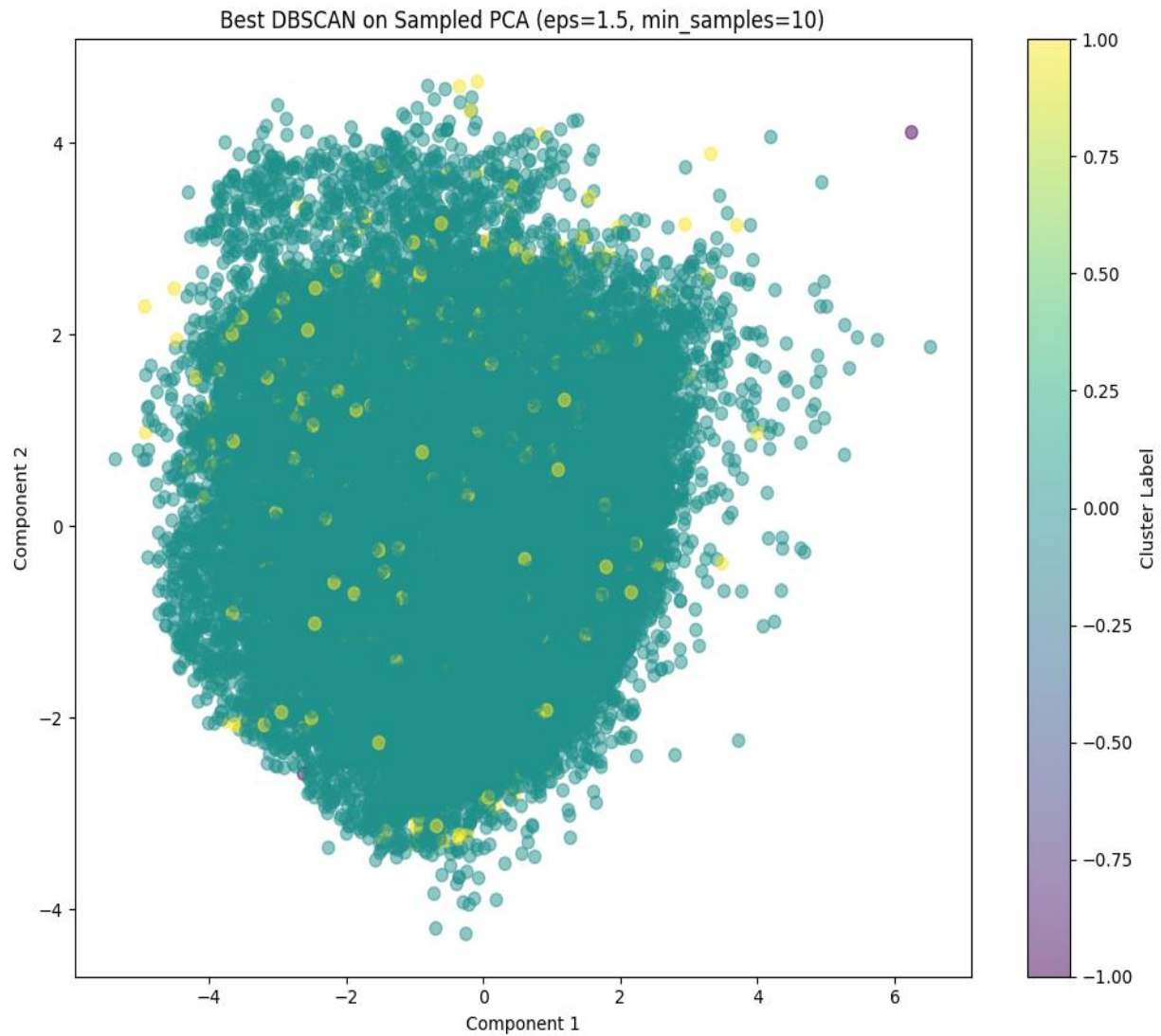
Best K-Means on Sampled PCA (n_clusters=4)





Best DBSCAN on Sampled PCA (eps=1.5, min_samples=10)





Silhouette Score Comparison

	Original Data	PCA Data
K-Means	0.1856	0.3077
DBSCAN	0.2804	0.5602

Clustering Results and Profiling

1. Best K-Means on Sampled PCA

1.1 Numerical Feature Means per Cluster

Cluster	Manufacturing Year	Engine HP	Engine CC	Kilometers	Price (€)	No. of Doors
0	2012.45	138.77	656.39	207,020	6,274	3.65
1	2009.96	136.09	859.95	228,517	4,679	5.01
2	2016.26	111.80	417.33	105,408	8,944	4.92
3	2016.88	191.42	1053.05	151,436	19,654	4.80

- **Cluster 1** seems to represent **older, high-mileage, and lower-priced vehicles**.
- **Cluster 2** groups **newer, economical vehicles** with small engines and moderate prices, possibly city cars or budget models.
- **Cluster 3** clusters **newer, high-performance cars** with high engine power and prices, likely premium or luxury vehicles.
- **Cluster 0** is somewhat in between, representing **average vehicles** in terms of age and price, but with fewer doors.

1.2 Top 3 Categorical Feature Distributions per Cluster

Make Distribution:

Cluster	Top 3 Makes
0	Volkswagen, BMW, Audi
1	Volkswagen, Renault, BMW
2	Dacia, Renault, Volkswagen
3	BMW, Mercedes, Audi

Fuel Type Distribution:

Cluster	Top 3 Fuel Types
---------	------------------

0	Diesel, Hybrid (U), Gasoline (G)
1	Diesel, Hybrid (U), Gasoline (G)
2	Diesel, Hybrid (U), Electric (E)
3	Diesel, Hybrid (U)

Transmission Type Distribution:

Cluster	Top 3 Transmissions
0	Manual, Automatic, Missing
1	Manual, Missing, Automatic
2	Manual, Missing, Automatic
3	Automatic, Manual, Missing

Body Type Distribution:

Cluster	Top 3 Body Types
0	Berlina, Hatchback, Break
1	Break, Berlina, Hatchback
2	SUV, Hatchback, Berlina
3	SUV, Berlina, Sedan

- **Cluster 2** (economical cars) has a strong presence of Dacia and Renault, typically affordable brands in Europe.

- **Cluster 3** is dominated by luxury brands like BMW, Mercedes, and Audi, with a higher percentage of automatic transmissions and SUVs or sedans.
- **Cluster 1** consists largely of older station wagons (Break) and manual transmissions, which supports the budget/family use hypothesis.

2. Best DBSCAN on Sampled PCA

2.1 Numerical Feature Means per Cluster

Cluster	Manufacturing Year	Engine HP	Engine CC	Kilometers	Price (€)	No. of Doors
-1 (Noise)	2003.67	143.00	1799.50	66,668	17,817	6.00
0	2013.16	141.74	772.68	179,232	9,287	4.74
1	2012.39	150.78	801.31	165,741	9,588	2.00

- **Cluster 0** represents the **majority of the data**, with balanced average values across features.
Cluster 1 seems to contain **sporty or compact vehicles**, likely **coupes or convertibles** given the 2-door average.
- **Noise Cluster (-1)** contains a few **outliers** (e.g., rare makes like MG, Fiat), but interestingly, the average price is quite high, possibly due to small sample size.

2.2 Top 3 Categorical Feature Distributions per Cluster

Make Distribution:

Cluster	Top 3 Makes
-1	MG, FIAT (rare entries)
0	Volkswagen, BMW, Renault
1	Mercedes, BMW, Audi

Fuel Type Distribution:

Cluster	Top 3 Fuel Types
-1	Electric, Hybrid, Diesel
0	Diesel, Hybrid
1	Diesel, Hybrid, Electric

Transmission Type Distribution:

Cluster	Top 3 Transmissions
-1	Manual, Automatic
0	Manual, Automatic, Missing
1	Manual, Automatic, Missing

Body Type Distribution:

Cluster	Top 3 Body Types
-1	SUV, Pickup
0	Berlina, SUV, Hatchback
1	Coupe, Cabrio, Monovolum

- **DBSCAN** effectively captured a **sporty/compact cluster (Cluster 1)** with **low door count**, high HP, and brands like **Mercedes, BMW, Audi**.
- The **main cluster (0)** resembles an overall balanced profile of the dataset.
- **Noise cluster (-1)** consists of **uncommon vehicles**, possibly luxury or niche models. These are valid outliers for a marketplace dataset.

Evaluation

- **Silhouette Score** was used to evaluate the clustering quality. While K-Means provided clear cluster separation and balanced cluster sizes, it required specifying the number of clusters in advance and assumed spherical shapes.
- **DBSCAN**, on the other hand, identified **dense groupings** and **handled outliers** more effectively without a pre-specified number of clusters. However, it formed fewer distinct clusters and is sensitive to parameter tuning (eps, min_samples).
- **K-Means** offered more interpretable clusters in this dataset, especially for segmenting vehicle listings by market tier (budget vs. premium).
- **DBSCAN** was useful in identifying **specialized segments** (coupes, sports cars) and **outliers**, which could be relevant for anomaly detection or niche marketing.
- The combination of **PCA for dimensionality reduction** followed by clustering provided meaningful insights into vehicle patterns across various technical and categorical attributes.